Projet PHP Trombinoscope

Prérequis : Bases HTML, PHP procédural, notions de SQL

Présentation

Vous disposez d'un site statique HTML entièrement fonctionnel visuellement. Votre travail consiste à le rendre dynamique en PHP et MySQL. Chaque page HTML a son équivalent PHP à créer. Vous ne repartez pas de zéro : la structure, le CSS et les pages de maquette sont fournis.

Le projet est une application de trombinoscope. Les membres peuvent s'inscrire, se connecter, consulter les profils de leurs camarades, publier des messages sur leur profil et commenter les publications des autres.

Parcourez attentivement les fichiers HTML fournis. Ils contiennent des indices sur la structure de la base de données, les champs à créer dans les formulaires, et les fonctionnalités à implémenter.Cela vous permettra de bien comprendre la structure de l'application avant de commencer à coder.

Structure du projet fourni

```
└── 📁 trombinoscope
  └── 📁 assets
    └── 📁 css
      ├── style.css
    └── 📁 images
    └── 📁 js
      ├── script.js
    └── 📁 uploads
  ├── edit-post.html
  ├── edit-profil.html
  ├── index.html
  ├── login.html
  ├── profil.html
  ├── register.html
  └── SUJET.pdf
```

Chaque fichier `.html` deviendra un fichier `.php`. Commencez toujours par renommer la page sur laquelle vous travaillez, puis adaptez son contenu.Pensez à utiliser l'inclusion de fichiers pour éviter les répétitions (ex : `config.php` pour la connexion à la base, `header.php` pour l'en-tête, `footer.php` pour le pied de page, etc...).

(La sécurité n'est pas un choix mais une obligation. Appliquez systématiquement les bonnes pratiques vues en cours, même si elles ne sont pas explicitement demandées dans le sujet. Ne faites jamais de compromis sur la sécurité, même pour gagner du temps.)

Base de données

Créez une base de données nommée `trombinoscope` et exécutez le script SQL se trouvant dans `db.sql` pour créer les tables. Lisez-le attentivement : il vous donne la structure de données avec laquelle vous travaillerez pendant tout le projet.

Fichier de connexion à la base de données

Avant toute chose, créez un fichier `config.php` qui établit la connexion PDO à votre base de données. Ce fichier sera inclus dans toutes les pages qui en ont besoin avec `require_once` .

Utilisez PDO avec le mode d'erreur `PDO::ERRMODE_EXCEPTION` . Cherchez dans la documentation PHP comment instancier une connexion PDO et définir son mode d'erreur.

Partie 1 : Inscription

Renommez `register.html` en `register.php` .

En haut du fichier, avant tout HTML, traitez le formulaire si la requête est en POST. Récupérez chaque champ avec `$_POST` . Vérifiez qu'aucun champ obligatoire n'est vide. Vérifiez que l'email n'existe pas déjà dans la table `utilisateurs` en effectuant une requête SELECT avec PDO.

Si une erreur est détectée, stockez le message d'erreur dans `$_SESSION['flash_error']` et redirigez vers `register.php` . Si tout est valide, hachez le mot de passe avant de l'enregistrer. La fonction à utiliser est `password_hash()` avec l'algorithme `PASSWORD_DEFAULT` . N'enregistrez jamais un mot de passe en clair.

Pour l'upload de photo, consultez la Partie 2 de ce sujet. Vous pouvez commencer sans la gestion de l'image et l'ajouter dans un second temps.

Une fois l'inscription réussie, stockez un message de succès dans `$_SESSION['flash']` et redirigez vers `login.php` .

Pour afficher les messages flash dans le HTML, ajoutez ce bloc juste après `session_start()` :

> Gestion des messages flash

- Lire $_SESSION['flash'] ou $_SESSION['flash_error']
- Le stocker dans une variable locale
- Le supprimer de la session avec unset()
- L'afficher conditionnellement dans le HTML la variable locale.

Ce pattern est le même sur toutes les pages. Appliquez-le systématiquement.

> Pensez à inclure ce bloc dans toutes les pages où vous souhaitez afficher des messages flash, pas seulement *`register.php`* .

Partie 2 : Upload de photo de profil

L'upload de fichier en PHP passe par `$_FILES`. Quand un formulaire contient un champ `<input type="file">`, les informations sur le fichier envoyé sont accessibles dans `$_FILES['avatar']`. Mais pour que cela fonctionne, il faut que le formulaire ait l'attribut `enctype="multipart/form-data"`.

Avant de déplacer le fichier, vous devez effectuer trois vérifications.

Vérifiez que le fichier a bien été reçu sans erreur en contrôlant `$_FILES['avatar']['error']`. La valeur `UPLOAD_ERR_OK` indique que tout s'est bien passé.Afin de bien comprendre les différentes erreurs possibles et la structure de `$_FILES`, faites un `var_dump($_FILES)` lors du développement pour voir exactement quelles données sont disponibles.

Vérifiez le type MIME du fichier. Utilisez `mime_content_type()` sur le chemin temporaire `$_FILES['avatar']['tmp_name']` et autorisez uniquement les formats d'image courants : `jpeg`, `png`, `webp` et `avif`. Refusez les autres types de fichiers.

Vérifiez la taille avec `$_FILES['avatar']['size']`. Refusez les fichiers supérieurs à 2 Mo (2 097 152 octets).

Si ces trois contrôles sont passés, générez un nom de fichier unique avec `uniqid()` suivi de l'extension extraite du nom d'origine.

> lire la documentation de *`uniqid()`* pour comprendre comment elle fonctionne et pourquoi elle est adaptée pour générer des noms uniques. Pour extraire l'extension, vous pouvez utiliser *`pathinfo(NOM DE VOTRE FICHIER, PATHINFO_EXTENSION)`*. Déplacez le fichier depuis son emplacement temporaire vers le dossier *`uploads/`* avec la fonction *`move_uploaded_file()`*.

Si aucun fichier n'est envoyé ou si une erreur survient, conservez la valeur `default.svg` comme avatar.

Partie 3 : Connexion et session

Renommez `login.html` en `login.php`.

Traitez le formulaire en POST. Recherchez l'utilisateur dans la base par son email. Si aucun utilisateur n'est trouvé, affichez un message d'erreur générique sans préciser si c'est l'email ou le mot de passe qui est incorrect. Cette précision aiderait un attaquant.

Si l'utilisateur existe, vérifiez le mot de passe avec `password_verify()`. En cas d'échec, même message générique tout en gardant l'email pré-rempli dans le formulaire pour faciliter la correction.

Si la connexion est valide, stockez dans la session au minimum l'identifiant et le prénom de l'utilisateur. Ces deux informations vous suffiront pour la majorité des opérations.

Gérez ensuite la case "Se souvenir de moi". Si elle est cochée, créez un cookie nommé `remember_email` contenant l'email de l'utilisateur, avec une durée de vie de 30 jours. Sur la page `login.php`, si ce cookie existe, pré-remplissez le champ email. Notez que le cookie ne doit contenir que l'email, jamais le mot de passe.

Après connexion, redirigez vers `index.php`.

Partie 4 : Protection des pages et déconnexion

Créez un fichier `auth.php` . Il contient uniquement ces quelques lignes : démarrer la session, vérifier si `$_SESSION['user_id']` existe, et si ce n'est pas le cas, stocker un message flash et rediriger vers `login.php` .

Toutes les pages qui nécessitent d'être connecté commenceront par `require_once 'auth.php'` à la place de `session_start()` .

Les pages protégées sont : `profil.php` , `edit-profil.php` , `edit-post.php` .

Créez `logout.php` . Ce fichier ne contient pas de HTML. Il vide la session, détruit le cookie de session côté navigateur, appelle `session_destroy()` , puis redirige vers `index.php` . Cherchez dans la documentation comment supprimer correctement un cookie avec `setcookie()` .

Partie 5 : Accueil et liste des membres

Renommez `index.html` en `index.php` . Cette page est publique, elle ne nécessite pas d'être connecté.

Récupérez tous les utilisateurs depuis la base avec un SELECT et affichez-les dans la grille. Chaque carte doit être un lien vers `profil.php?id=X` où X est l'identifiant de l'utilisateur.

Gérez le filtre par promotion. L'URL `index.php?promo=BUT1+2024` doit n'afficher que les membres de cette promotion. Si aucun filtre n'est passé, tous les membres s'affichent. Utilisez `$_GET['promo']` et ajustez votre requête SQL avec une clause `WHERE` conditionnelle.

Partie 6 : Page de profil

Renommez `profil.html` en `profil.php` . Cette page est publique mais certains éléments ne s'affichent que si l'utilisateur connecté est propriétaire du profil.

Récupérez l'identifiant dans l'URL via `$_GET['id']` . Si cet identifiant est absent ou ne correspond à aucun utilisateur, redirigez vers `index.php` .

Récupérez également toutes les publications de cet utilisateur, triées de la plus récente à la plus ancienne, ainsi que les commentaires associés à chaque publication en faisant une Requête SQL avec jointure cf. la documentation MySQL sur les jointures `sql.sh`

Pour chaque publication, vérifiez si `$_SESSION['user_id']` correspond à l'auteur de la publication. Si c'est le cas, affichez les boutons Modifier et Supprimer. Sinon, ne les affichez pas. C'est le contrôle de propriété.

Affichez de même le formulaire d'ajout de publication uniquement si le visiteur est connecté et consulte son propre profil.

Affichez le formulaire de commentaire uniquement si le visiteur est connecté.

Partie 7 : Publications

Créez `post.php` . Ce fichier ne contient pas de HTML. Il est appelé depuis le formulaire de la page profil.

Commencez par inclure `auth.php` : seul un utilisateur connecté peut publier. Récupérez le contenu du formulaire, vérifiez qu'il n'est pas vide, insérez la publication en base en associant l'identifiant de l'utilisateur connecté, puis redirigez vers `profil.php?id=X` .

Créez `edit-post.php` . Ce fichier traite le formulaire de modification. Avant d'effectuer le UPDATE, vérifiez que la publication appartient bien à l'utilisateur connecté. Pour cela, effectuez un SELECT de la publication, comparez son champ `utilisateur_id` avec `$_SESSION['user_id']` . Si les identifiants ne

correspondent pas, redirigez sans effectuer aucune modification. C'est le contrôle de propriété côté serveur.

Créez `delete-post.php` . Même logique : récupérez l'identifiant de la publication depuis `$_GET['id']` , vérifiez la propriété, puis effectuez le DELETE. Redirigez vers le profil après suppression.

Renommez `edit-post.html` en `edit-post.php` . Récupérez l'identifiant de la publication dans l'URL, vérifiez la propriété, et pré-remplissez le formulaire avec le contenu actuel.

Partie 8 : Commentaires

Créez `comment.php` . Ce fichier ne contient pas de HTML.

Incluez `auth.php` . Récupérez l'identifiant de la publication depuis `$_POST['post_id']` et le contenu du commentaire. Vérifiez que la publication existe bien avant d'insérer le commentaire. Insérez le commentaire avec l'identifiant de l'utilisateur connecté. Redirigez vers la page de profil de l'auteur de la publication.

Partie 9 : Modification du profil

Renommez `edit-profil.html` en `edit-profil.php` . Cette page est protégée par `auth.php` .

Lors de l'affichage de la page, pré-remplissez tous les champs avec les données actuelles de l'utilisateur connecté, récupérées depuis la base.

Lors du traitement POST, vérifiez que l'utilisateur modifie bien son propre profil en comparant `$_SESSION['user_id']` avec l'identifiant traité. Un utilisateur ne peut jamais modifier le profil d'un autre.

Construisez la requête UPDATE en incluant uniquement les champs à modifier. Pour le mot de passe, n'effectuez la mise à jour que si le champ n'est pas vide, en le hachant avec `password_hash()` . Pour l'avatar, appliquez la logique d'upload décrite en Partie 2.

Après modification, mettez à jour les informations dans la session si nécessaire (par exemple le prénom affiché dans la navigation), puis redirigez vers `profil.php?id=X` avec un message flash de succès.

Récapitulatif des fichiers PHP à créer

Fichier	Rôle
`config.php`	Connexion PDO
`auth.php`	Protection des routes
`register.php`	Inscription + upload avatar
`login.php`	Connexion + cookie remember
`logout.php`	Déconnexion
`index.php`	Liste des membres + filtre promo
`profil.php`	Profil + publications + commentaires
`edit-profil.php`	Modification du profil
`post.php`	Créer une publication
`edit-post.php`	Modifier une publication
`delete-post.php`	Supprimer une publication
`comment.php`	Ajouter un commentaire

Points de vigilance

Toujours utiliser `htmlspecialchars()` avant d'afficher une donnée issue de la base ou d'un formulaire.

Toujours utiliser des requêtes préparées PDO avec des paramètres liés. Ne jamais concaténer directement une variable dans une requête SQL.

Toujours mettre `exit` après un `header('Location: ...')` .

Ne jamais faire confiance à ce qui vient du navigateur : vérifiez côté serveur que l'utilisateur a bien le droit d'effectuer une action, même si le bouton est masqué dans l'interface.

Bonus 1: pagination et CSRF

Si vous avez terminé toutes les fonctionnalités demandées, vous pouvez ajouter une pagination sur la page d'accueil pour n'afficher que 10 membres par page. Utilisez `LIMIT` et `OFFSET` dans votre requête SQL, et ajoutez des liens de navigation pour passer d'une page à l'autre.

Pour protéger les formulaires contre les attaques CSRF, générez un token aléatoire lors de l'affichage du formulaire, stockez-le dans la session, et ajoutez-le dans le formulaire en tant que champ caché. Lors du traitement du formulaire, vérifiez que le token soumis correspond à celui stocké en session. Si ce n'est pas le cas, refusez la requête.

Bonus 2: SQL Requêtes Préparées

Si vous avez terminé toutes les fonctionnalités demandées, essayez de réécrire toutes vos requêtes SQL en utilisant des requêtes préparées PDO avec des paramètres liés. Cela renforcera la sécurité de votre application contre les injections SQL et vous permettra de maîtriser cette technique essentielle en PHP.